

ZaraiLink: Demo

Deep Dive

An exhaustive, beginner-safe walkthrough of the public demo

Read this before anything else

This document describes a demo, not a product. Every company, contact, phone number, email address, price and shipment figure in this repository is **fabricated**. Nothing here is real trade data. No part of this code talks to a network. It is a hand-built imitation of another application's screens.

ZaraiLink itself is a team project, not solo work. It was built by three people. Salman Adnan led it; Umar Kashif and Fahad Nadeem wrote real and substantial parts of it. The real system is private and its source is not in this repository. This demo exists so the product can be shown without publishing code that three people would have to agree to publish.

Contents

1	What this repository is, and what it is not	3
1.1	The one-paragraph version	3
1.2	The two framings that must never be dropped	3
1.3	The relationship between the two repositories	4
2	Layout of the repository	4
2.1	Every file, and why it exists	5
2.2	How the browser loads it	5
3	The data: what was invented, and how it is shaped	6
3.1	Why the shape matters more than the values	6
3.2	Dead data	7
4	The router: six screens in one file	8
4.1	How the screens work	8
4.2	Application state	9
4.3	One listener for the whole application	10
5	The Dashboard and the search-mode detector	11
5.1	What the screen does	11
5.2	A branch that can never be reached	11
5.3	Running a search	12
6	The Data Dashboard: results, pills and the paywall	13
6.1	The sidebar filters	13
6.2	The four trade-direction pills	13
6.3	The category paywall, and a real bug	14
6.4	The locked result cards	15
7	The Trade Directory and the Watchlist	15
7.1	Rendering the grid	15
7.2	The watchlist, and a real state bug	16
7.3	The star icon trick	16
8	The Company Profile: masking, seniority, and the unlock flow	16
8.1	Masking	16
8.2	The seniority classifier	17
8.3	The unlock flow	18
8.4	The token economy, end to end	20
9	Presentation: icons, fonts and colour	21
9.1	The icon set	21
9.2	The font, and a stale README	21
9.3	Colour tokens	22
10	Security, licensing and deployment	23
10.1	XSS, and why it does not matter here	23
10.2	Deployment	24
10.3	Licence	24

11 Consolidated findings	24
11.1 Real defects	25
11.2 Dead code and dead controls	25
11.3 Where the README overstates the code	25
11.4 What the code does well	25
11.5 The most likely interview questions	26

1 What this repository is, and what it is not

1.1 The one-paragraph version

This repository, `zarailink-demo`, is a **static web page**: three JavaScript files, one stylesheet, one HTML file, a font and two favicons. Opening `index.html` in a browser gives you a clickable imitation of six screens from a different, private application called ZaraiLink. You can type a search, filter results, browse a company directory, open a company profile, spend fake tokens to reveal fake phone numbers, and star companies onto a watchlist. Every one of those actions is computed locally from a hard-coded array of invented companies. There is no server, no database, no build step, and no network request of any kind.

Why a demo exists at all

The real ZaraiLink repository is private, and it cannot be made public by one person. It has three authors, and publishing shared work is not a decision one author gets to make alone. That leaves a problem: how do you show a hiring panel a product they cannot be given the source to? The answer taken here is to rebuild the *screens* against *invented* data. The visual design language and the interaction patterns are things any browser loading the real app already downloads and renders. The private parts (the natural-language pipeline, the ranking model, the customs data, the API routes) stay out of this repository entirely.

1.2 The two framings that must never be dropped

Both of the following are stated in the repository's own `README.md`, its `index.html` banner and its footer, and both are load-bearing. If either is dropped when this project is presented, the presentation becomes dishonest.

1. **The data is fabricated.** “Meridian AgroTrade Ltd.” does not exist. Neither does “Elena Marsh” or her phone number. The domains are all under `.example.com`, a reserved domain that can never resolve to a real site. The headline statistics (“50,000+ Trade Records”) are invented strings, not counts of anything.
2. **ZaraiLink is a team project led by Salman Adnan.** The demo shows features from a shared codebase. Some of those features were written by other people. This demo demonstrates that the features work as described; it does not claim their authorship.

What this repository cannot prove, and should not be asked to

This demo is evidence of *UI reconstruction skill and product judgment*. It is **not** evidence that the real ZaraiLink's backend works, because none of that backend is here. A reader of this repository alone cannot verify that the natural-language pipeline, the hybrid retrieval, or the learning-to-rank model exist or perform. Those claims rest entirely on the owner's word and on the private repository.

The specific claim in `README.md` that Salman wrote “64 of 97 commits” is **unverifiable from this repository**. The private repository's git history is not here. That number is the owner's to stand behind, and an interviewer who asks for it should be pointed at the private repository, not at this one.

1.3 The relationship between the two repositories

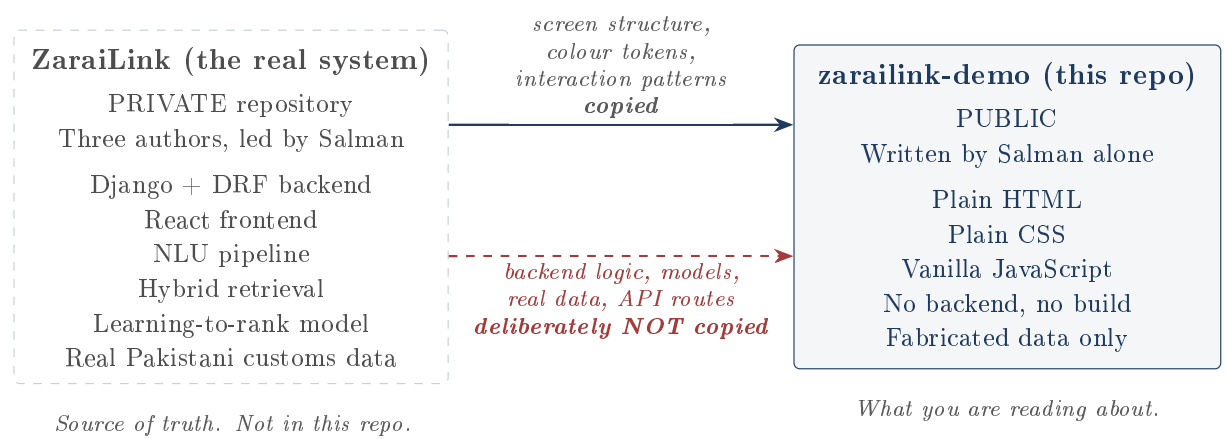


Figure 1: The demo copies the visible surface and deliberately copies none of the private logic.

Jargon: Backend and frontend

A **frontend** is the part of an application that runs in the user’s browser and draws what you see. A **backend** is the part that runs on a server somewhere else, holds the database, and answers the frontend’s questions over the network. The real ZaraiLink has both. This demo has *only* a frontend, and that frontend asks nobody anything: it reads a JavaScript array sitting in the same folder.

2 Layout of the repository

2.1 Every file, and why it exists

```

zarailink-demo/
├── index.html all six screens, as static markup
├── styles.css colour tokens and component layout
├── demo-data.js the fabricated companies and plans
├── icons.js a hand-drawn SVG icon set
├── app.js all behaviour: routing, filtering, unlocking
├── assets/
│   └── fonts/
│       └── inter-variable.woff2 self-hosted font, 48 KB
├── favicon.svg emerald tile with a Z
├── favicon.png raster fallback
├── netlify.toml two lines: publish this folder as-is
├── LICENSE PolyForm Strict 1.0.0
├── README.md
├── docs/
│   └── explainers/
│       ├── deep-dive.tex this document
│       └── brief.tex

```

Figure 2: The whole project. Five files carry all of the behaviour.

The entire application is roughly 1,100 lines across five files:

File	Role	Lines
app.js	All behaviour and rendering	399
styles.css	All presentation	369
index.html	Static shell for six screens	271
demo-data.js	The fabricated dataset	70
icons.js	SVG icon lookup table	34

2.2 How the browser loads it

There is no build step. This deserves emphasis, because it is unusual for something that imitates a React application.

Jargon: Build step

Most modern web applications are written in a form the browser cannot run (React’s JSX, TypeScript, Sass), and a **build step** is a program that translates that source into plain HTML, CSS and JavaScript before it is deployed. It is a whole extra machine to install, configure, run and keep working. This demo skips it: what is in the repository is exactly what the browser executes. You can double-click `index.html` and it works, offline, forever, with no toolchain.

The three scripts are loaded in a fixed order at the bottom of `index.html`, and that order matters:

```
1 <script src="demo-data.js"></script>
2 <script src="icons.js"></script>
3 <script src="app.js"></script>
```

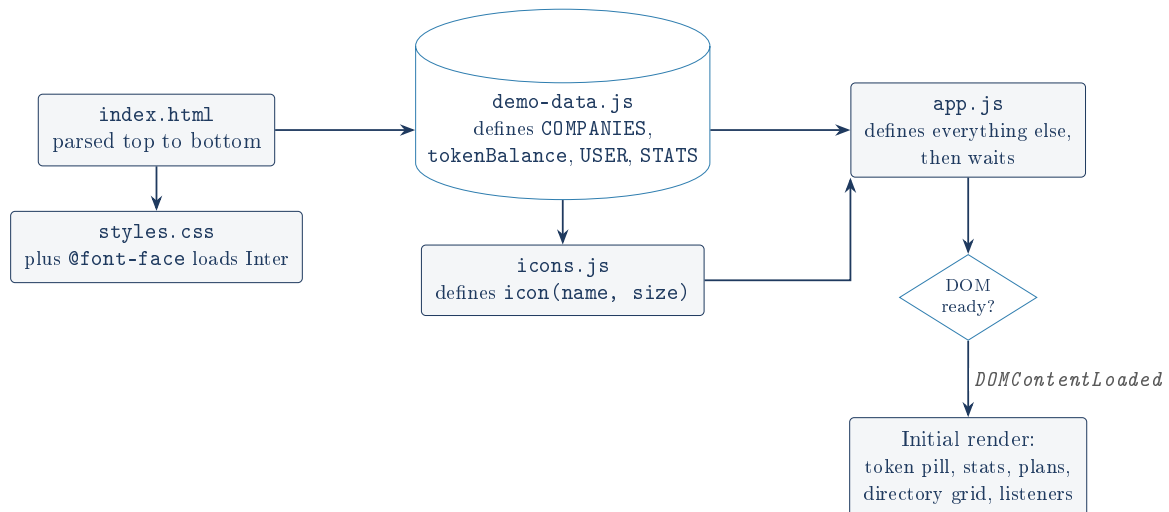


Figure 3: Load order. `app.js` depends on globals that the two earlier scripts define.

The load order is an unguarded dependency

`app.js` uses `COMPANIES`, `tokenBalance` and `icon()` as bare globals. Nothing declares or checks that dependency. If someone reorders those three `<script>` tags, or adds `defer` to only one of them, the page breaks with a `ReferenceError` and no useful message. For a five-file demo that is an acceptable trade (a module system would be more machinery than the problem deserves), but it is a real coupling that lives only in the ordering of three lines of HTML, and it is worth naming rather than pretending the files are independent.

3 The data: what was invented, and how it is shaped

3.1 Why the shape matters more than the values

The values in `demo-data.js` are invented and worthless. The *field names* are the interesting part: they mirror the real application's components, so the demo exercises the real product's actual information architecture rather than a simplified stand-in.

```

COMPANIES array of 4
├── id "c1" .. "c4", used as the routing key
├── name / legal_name display name vs registered entity
├── country / district / province drives the trade-direction pills
├── sector / type / established / employees profile stat grid
├── verified / has_key_contacts both needed for the Verified badge
├── website / last_active PRESENT BUT NEVER READ
├── products array
│   └── name / variety / value_added
├── avg_price / volume / shipments / avg_shipment result-card stats
└── contacts array, 1 to 2 per company
    ├── id "k1" .. "k5", the unlock key
    ├── name / designation
    ├── is_public free, never locked
    ├── unlocked MUTATED at runtime
    └── phone / email / whatsapp whatsapp may be null

```

Figure 4: The company record. Two fields are decorative: nothing ever reads them.

The full fabricated dataset is small enough to state exactly:

Company	Country	Sector	Verified	Contacts
Meridian AgroTrade Ltd.	Brazil	Sweeteners	yes	2
Cascade Commodities Group	Brazil	Sweeteners	yes	1 (public)
Sable Harvest Exports	Brazil	Sweeteners	no	1
Northgate Chemical Traders	Pakistan	Fertilizers	yes	1

Four companies, five contacts. One of those five (Priya Nair at Cascade) is marked `is_public: true`, so it starts unlocked and free. The other four cost one token each. The user starts with `tokenBalance = 7`.

The dataset is smaller than the screens it has to fill

Three of the four companies are Brazilian and sell dextrose; one is Pakistani and sells urea. That is enough to make the country-based filtering visibly do *something*, but only just. As shown later, it means two of the four trade-direction pills return three results and the other two return one result, and it means no screen ever shows enough rows for the sidebar filters or the grid layout to be stress-tested. A panel clicking through this will notice the world is thin.

3.2 Dead data

Three declarations in `demo-data.js` are never read by anything. This was verified by searching every file in the repository for each identifier.

Declaration	Looks like	Actually
HS_TREE	a lookup of 3 HS codes	declared, referenced nowhere
company.website	a profile link	set on all 4, rendered never
company.last_active	a recency signal	set on 3, rendered never

Jargon: HS code

The Harmonized System is an international numbering scheme for traded goods. Every physical product that crosses a border has an HS code, and customs records are indexed by it. 1702.3000 is dextrose. The real ZaraiLink lets a user search by this code, which is why the demo's search box detects a leading digit and says so.

HS_TREE is the most interesting of the three, because it is a real fragment of the product's model (code, name, category) that got built and then never wired to a screen. The Dashboard's "4 Ways to Search" card hard-codes the example 1702.3000 in the HTML instead of reading it from HS_TREE, and `runSearch()` hard-codes the string "HS 1702.3000: Dextrose" rather than looking the code up. The data structure that would make that dynamic exists and is ignored.

4 The router: six screens in one file

4.1 How the screens work

All six screens are present in `index.html` at once, as six `<section class="view">` elements. CSS hides all of them, and shows only the one carrying an extra class:

```
1 .view { display: none; }
2 .view.is-active { display: block; }
```

Switching screens is therefore just moving one CSS class:

```
1 function switchView(viewId) {
2   document.querySelectorAll(".view").forEach(v =>
3     v.classList.toggle("is-active", v.id === `view-${viewId}`));
4   document.querySelectorAll(".zl-navbar__links > a").forEach(a =>
5     a.classList.toggle("is-active", a.dataset.view === viewId));
6   window.scrollTo({ top: 0, behavior: "smooth" });
7 }
```

The two-argument form of `classList.toggle` is the trick that makes this three lines instead of ten: `toggle(name, condition)` adds the class when the condition is true and removes it when false, so one pass over every view both shows the target and hides all the others.

Jargon: The DOM, and `querySelectorAll`

The **DOM** (Document Object Model) is the browser's live, in-memory tree of the page. JavaScript changes what you see by changing that tree. `querySelectorAll(".view")` hands back every element whose class is `view`, using the same selector syntax CSS uses.

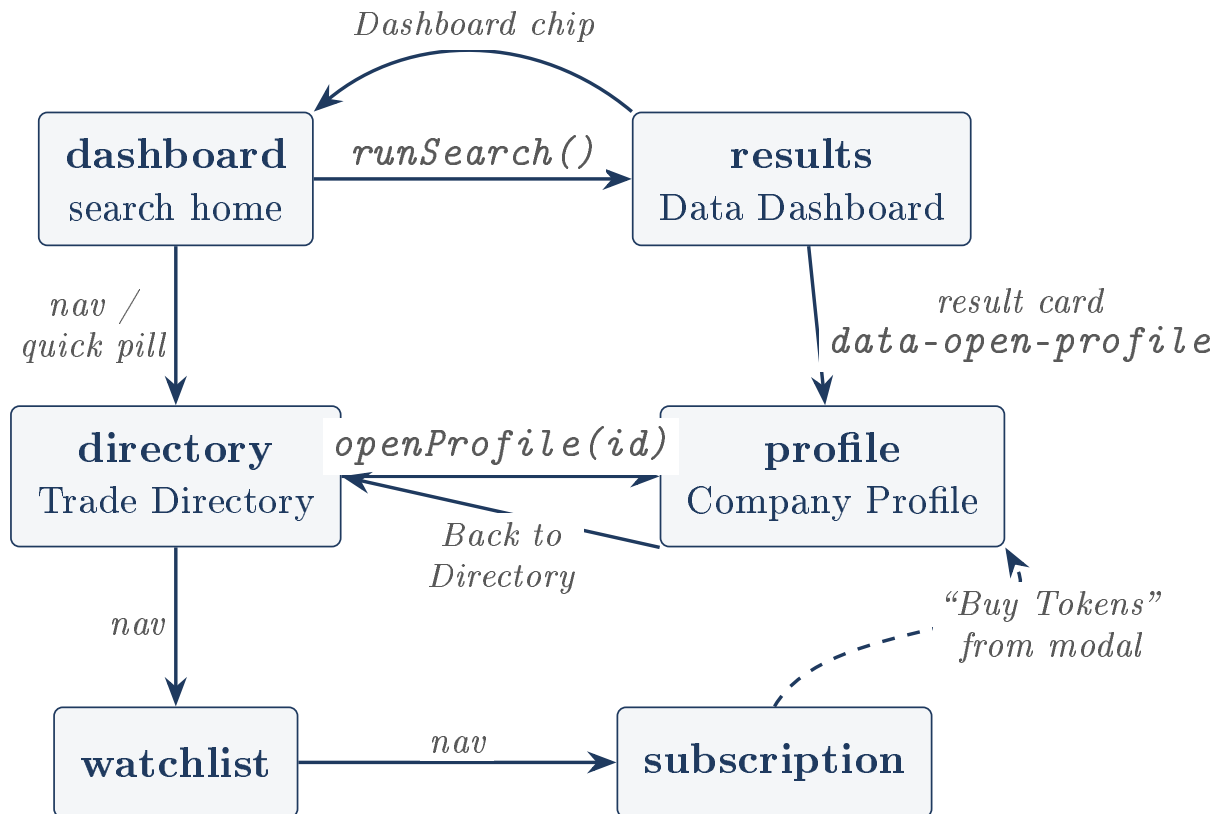


Figure 5: The navigation graph. Every edge is a class swap; no URL ever changes.

No URL routing, and what that costs

`switchView` never touches the address bar. There is no `history.pushState`, no hash fragment, no router. Three consequences, all real:

- **The browser Back button leaves the demo.** It does not go back a screen; it leaves the page entirely. The only way back is the in-page “Back to Directory” link.
- **No screen is linkable.** You cannot send someone a URL that opens a company profile. For a portfolio demo whose entire job is to be shown to other people, that is a genuine limitation.
- **Reload resets everything.** State lives in plain variables, so a refresh restores 7 tokens and an empty watchlist.

For a scripted walkthrough this is defensible and the simplicity is worth something. But “I skipped the router deliberately” is a much better answer than being surprised by the question.

4.2 Application state

The entire state of the application is six module-level variables:

```

1 const watchedIds = new Set(); // starred company ids
2 let currentScope = "import"; // Import | Export      <- never read
3 let currentRole  = "Supplier"; // Supplier | Buyer
4 let currentProfileId = null; // which company the profile shows
5 let ddUnlocked   = false;    // category payroll cleared?
6 let ddActivePill = null;     // which trade-direction pill
7 let pendingUnlock = null;    // { companyId, contactId } awaiting confirm

```

plus `tokenBalance`, which lives in `demo-data.js` and is mutated from `app.js`. There is no state container, no framework, and no persistence.

tokenBalance is defined in the data file and mutated from the app file

`demo-data.js` declares `let tokenBalance = 7` alongside the fabricated companies, and `app.js` writes to it from three different places. Mixing “the fixtures” and “the mutable session state” in the file named `demo-data.js` is a small architectural smell: everything else in that file is inert data, and this one line is not. It works, but the file name lies slightly about its contents. Worse, the `unlocked` flag on each contact is mutated *in place* inside `COMPANIES`. The fabricated fixture and the live session state are the same object. It is why a reload is the only way to re-lock a contact.

4.3 One listener for the whole application

Rather than attaching a handler to every button, `app.js` attaches a single `click` listener to `document.body` and works out what was clicked:

```
1 document.body.addEventListener("click", (e) => {
2   const viewLink = e.target.closest("[data-view]");
3   if (viewLink) { /* navigate */ return; }
4   const openProfileBtn = e.target.closest("[data-open-profile]");
5   if (openProfileBtn) { /* open profile */ return; }
6   const watchBtn = e.target.closest("[data-watch]");
7   if (watchBtn) { /* toggle star */ return; }
8   // ... and so on
9 });
```

Jargon: Event delegation

When you click a button, the browser fires the event on the button, then on its parent, then its parent’s parent, all the way up to `<body>`. This is called **bubbling**. **Event delegation** exploits it: listen once at the top, and use `e.target.closest(selector)` to walk back *down* the chain and find which interesting element was actually hit. `closest()` starts at the clicked element and climbs until it finds an ancestor matching the selector, which is what makes clicking an SVG icon *inside* a button still register as clicking the button.

This is the single best decision in the codebase, and it is worth being able to explain why. Almost every element on these screens is created by writing an HTML string into `innerHTML`. Doing that **destroys and recreates** the elements, which would throw away any listener attached directly to them. With delegation, the listener lives on `<body>`, which is never recreated, so re-rendering a grid a hundred times costs nothing and breaks nothing. Without it, every render would have to re-attach every handler.

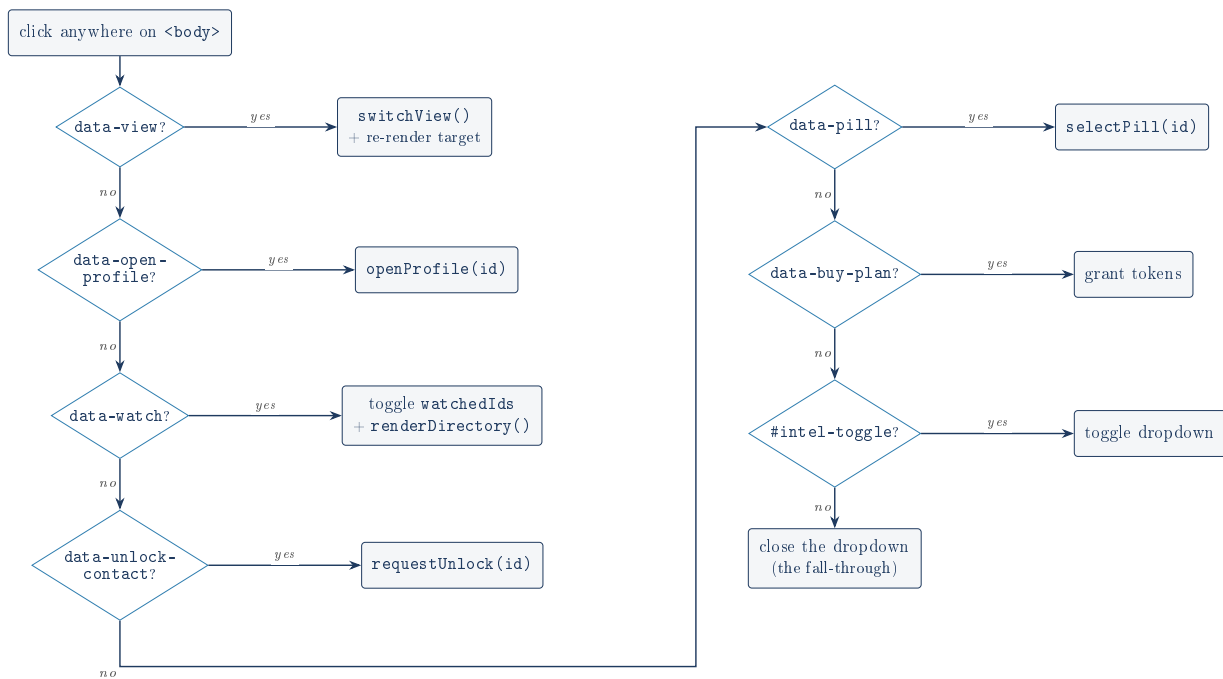


Figure 6: The delegated dispatcher, read left column first then right. Each **yes** branch ends in **return**, so it is an ordered chain, not a switch: the first matching test wins and nothing below it is consulted.

5 The Dashboard and the search-mode detector

5.1 What the screen does

The Dashboard is the real application’s post-login home. It carries a “4 Ways to Search” explainer card, a large search box, an Import/Export toggle, quick pills, three example queries, a dark statistics strip and a features grid.

The one piece of real logic on it is the **mode detector**: as you type, a coloured pill appears telling you how the query will be interpreted.

```

1 function detectMode(query, scopeSelected) {
2   if (/^\d/.test(query.trim())) return "hscode";
3   if (scopeSelected && query.trim()) return "ai";
4   return "product";
5 }

```

Jargon: Regular expression

`/^\d/` is a **regular expression**: a tiny pattern language for matching text. `^` means “at the very start”, and `\d` means “any digit”. So the whole pattern asks: does this string start with a digit? A query starting with a number is assumed to be an HS code.

5.2 A branch that can never be reached

“Product Search” mode is dead code, and the README claims otherwise

`detectMode` takes a `scopeSelected` argument, and its third branch returns “product” only when that argument is falsy. But **both** call sites pass the literal `true`:

```

1 const mode = detectMode(query, true); // in renderModePill
2 const mode = detectMode(q, true);    // in runSearch

```

Walk the branches with `scopeSelected` pinned to `true`:

- query starts with a digit → "hocode"
- query is non-empty and does not start with a digit → "ai"
- query is empty → "product", *but* `renderModePill` returns early on an empty query and clears the pill, so nothing renders.

Therefore "product" is **unreachable in the running application**. The green "Product Search" pill, and its `.mode-pill.is-product` CSS rule, can never appear. This was confirmed by executing the function directly rather than by reading it.

This matters because `README.md` states that typing a query "flips a colored pill between HS Code Mode, Product Search, and AI Query Mode". Two of those three are real. The third cannot happen. **The README overstates the feature.**

The cause is visible one line up: `currentScope` exists to hold whether Import or Export is selected, and `detectMode` clearly wants to receive it. Instead the call sites hard-code `true`. Passing `currentScope` would not fix it either, since `currentScope` is initialised to "import" and is therefore *always* truthy. The real design presumably starts the scope unselected (`null`) and only then does the distinction between product search and AI query mean anything.

The Import/Export toggle is decorative

`currentScope` is assigned in exactly one place and read in **none**:

```
1 document.querySelectorAll(".scope-btn").forEach(b => b.addEventListener("click", () => {
2   currentScope = b.dataset.scope;                                // written...
3   document.querySelectorAll(".scope-btn").forEach(x =>
4     x.classList.toggle("is-active", x === b));                    // ...only the class
5   matters
6 }));
```

Clicking Import or Export moves a green highlight and changes nothing else. Nothing in search, filtering or results consults it. The Dashboard card even tells the user "AI Query: Select Import/Export first", implying the toggle gates a behaviour. It does not.

5.3 Running a search

```
1 function runSearch(query) {
2   const q = (query || document.getElementById("query-input").value || "dextrose").trim();
3   const mode = detectMode(q, true);
4   ddUnlocked = false;
5   ddActivePill = null;
6   document.getElementById("dd-title").textContent =
7     mode === "hocode" ? "HS 1702.3000: Dextrose" : q;
8   document.getElementById("dd-sub").textContent =
9     "Sweeteners \u00B7 fabricated example results for this demo";
10  document.getElementById("dd-shipment-count").textContent =
11    `${COMPANIES.length * 14} shipments (demo)`;
12  renderFilters();
13  renderPills();
14  // ... prompt to pick a pill, then:
15  switchView("results");
16 }
```

Note the fallback chain: an explicit argument, else whatever is in the box, else the literal "dextrose". Searching with an empty box still produces a results screen.

The search query does not filter anything

`runSearch` computes `q`, and then uses it **only as a heading**. The result set is never filtered by it. Search “PVC Resin” and you get the same four dextrose and urea companies as searching “dextrose”, under a heading that says “PVC Resin”. Search any number at all and the heading hard-codes “HS 1702.3000: Dextrose” regardless of which number you typed.

The screen is honest at the level of a label (`dd-sub` reads “fabricated example results for this demo”) but the illusion is thin, and an interviewer who types two different searches will see it immediately. This is defensible as a scripted demo, but only if it is stated up front rather than discovered.

The shipment count is an arbitrary formula

`COMPANIES.length * 14` evaluates to **56**, and the chip reads “56 shipments (demo)”. The dataset’s own `shipments` fields sum to **125**. So the demo displays a headline count that contradicts its own data, computed by multiplying a company count by a magic number. `COMPANIES.reduce((a, c) => a + c.shipments, 0)` would have been both simpler to justify and internally consistent. The (demo) suffix carries a lot of weight here.

6 The Data Dashboard: results, pills and the paywall

6.1 The sidebar filters

`renderFilters()` builds the “Refine by Product” checkbox list by flattening every product name across every company:

```
1 const products = [...new Set(COMPANIES.flatMap(c => c.products.map(p => p.name)))];
```

Jargon: flatMap and Set

`map` turns each item into something else. `flatMap` does the same but then flattens one level of nesting, which is what you want when each company yields an *array* of products and you want one flat list. A `Set` is a collection that silently discards duplicates, so wrapping the list in `new Set(...)` and spreading it back out with `[...]` is the idiomatic JavaScript way to say “unique values”.

That yields three filters: Dextrose Monohydrate, Dextrose Anhydrous, Urea 46%.

The filter checkboxes are inert

`renderFilters` renders real `<input type="checkbox">` elements with real, correctly computed counts beside them, and **no listener is ever attached to them**. Ticking one does nothing. They are convincing precisely because the counts are right, which makes them the most misleading control on the page: a checkbox that visibly responds to the mouse and changes nothing reads as broken rather than as scripted.

6.2 The four trade-direction pills

```
1 const DD_PILLS = [
2   { id: "foreign-suppliers", label: "Foreign Suppliers", sub: "Import → Sellers",
3     filter: c => c.country !== "Pakistan" },
4   { id: "foreign-buyers", label: "Foreign Buyers", sub: "Export → Buyers",
5     filter: c => c.country !== "Pakistan" },
6   { id: "pk-buyers", label: "Pakistani Buyers", sub: "Import → Local Buyers",
7     filter: c => c.country === "Pakistan" },
```

```

8   { id: "pk-suppliers",      label: "Pakistani Suppliers", sub: "Export → Local Sellers",
9     filter: c => c.country === "Pakistan" }
10 ];

```

Storing the filter as a function on the data object is a neat touch: `selectPill` does not need to know what any pill *means*, it just calls `COMPANIES.filter(pill.filter)`.

Four pills, two behaviours

Read the predicates. “Foreign Suppliers” and “Foreign Buyers” have **identical** filters. So do “Pakistani Buyers” and “Pakistani Suppliers”. Verified by running them:

Pill	Predicate	Results
Foreign Suppliers	<code>country !== "Pakistan"</code>	3
Foreign Buyers	<code>country !== "Pakistan"</code>	3 (the same 3)
Pakistani Buyers	<code>country === "Pakistan"</code>	1
Pakistani Suppliers	<code>country === "Pakistan"</code>	1 (the same 1)

The supplier/buyer axis is **pure decoration**: only the country axis does anything. Clicking “Foreign Buyers” after “Foreign Suppliers” shows the identical three Brazilian companies, and the only visible difference is the result-count line, which reads “3 Buyers found” instead of “3 Suppliers found” because of this:

```
1  ${results.length} ${pill.label.includes("Buyer") ? "Buyers" : "Suppliers"} found
```

That is a string test on a display label to decide a noun, which also means it silently depends on the label never being reworded.

The demo’s `README.md` is careful here and says the pills “filter the same fabricated company set by country”, which is accurate. The screen is less careful than the `README`.

6.3 The category paywall, and a real bug

When a pill is selected and `ddUnlocked` is false, a banner is rendered below the results offering to unlock the whole category.

The paywall advertises 500 tokens and charges 5

This is a genuine defect, not a design choice. The button label and the arithmetic disagree by a factor of 100:

```

1  <button ... id="unlock-category-btn">Unlock for 500 Tokens</button>
2  ...
3  if (tokenBalance < 5) { showModal("modal-insufficient"); return; }
4  tokenBalance -= 5;

```

The user is told it costs 500. The code requires 5 and deducts 5. Since the user starts with 7 tokens, the button *works*, which is exactly why the bug survives: nothing appears broken. You click a button marked 500, you have 7, and it succeeds.

Worse, the cost 5 is a **magic number written twice**, once in the guard and once in the deduction, with the advertised price a third, unrelated literal in an HTML string. All three should be one constant. Compare the per-contact flow, which consistently uses 1 in the label, the guard and the deduction.

Which number is wrong is a judgment call. Against the plan grants (Starter gives 20 tokens for \$29/mo), a 500-token category unlock is absurd: no plan except Enterprise could ever afford one. So the *label* is almost certainly the typo and 5 is the intended price. Either way, one of the two must change.

6.4 The locked result cards

```
1 .dd-card.is-locked .dd-card__actions { filter: blur(2px); pointer-events: none; opacity: .6; }
```

Locked cards blur their action buttons and set `pointer-events: none`, which makes the element ignore the mouse entirely.

The blur is cosmetic, and here that is fine, but know why

A CSS blur hides nothing. The text is in the DOM, and anyone can read it in the developer tools or by deleting one class. In the real ZairaiLink, a paywall implemented this way would be a serious data leak: the paid-for contact details would be sitting in the page for anyone who pressed F12.

In *this* demo it is harmless, because every value behind the blur is fabricated and there is nothing to protect. But the pattern is only safe here for a reason that does not transfer, and the correct answer to “is this how the real one works?” is that a real paywall must withhold the data at the server and never send it.

7 The Trade Directory and the Watchlist

7.1 Rendering the grid

```
1 function renderDirectory() {
2   document.getElementById("td-title").textContent =
3     currentRole === "Supplier" ? "Find Suppliers" : "Find Buyers";
4   // ... lede and switch-button text likewise
5   const query = (document.getElementById("td-search-input").value || "").toLowerCase();
6   const results = COMPANIES.filter(c => c.name.toLowerCase().includes(query));
7   document.getElementById("directory-grid").innerHTML =
8     results.map(c => companyCardHtml(c)).join("")
9   || `<p style="color:var(--text-2)">No suppliers found.</p>`;
10 }
```

The directory search box *does* work, unlike the Dashboard’s: it filters on company name, case-insensitively, on every keystroke. Lowercasing both sides is the standard trick for a case-insensitive contains-match. Note the `||` fallback: `[] .join("")` produces an empty string, which is falsy, so the empty-state message appears automatically.

Three defects in one small function

1. **currentRole changes only nouns.** Switching to “Find Buyers” rewrites the heading, the lede and the button, and shows **the same four companies**. There is no buyer/supplier distinction in the data at all. The Trade Directory has exactly one mode wearing two hats.
2. **The empty state is hard-coded to “No suppliers found.”** Search for nonsense while in Buyer mode and it still says “suppliers”. The one string that was supposed to vary with `currentRole` is the one that does not.
3. **The search box is never cleared.** `renderDirectory` reads the input but nothing resets it. Type “Northgate”, navigate to a profile, come back via “Back to Directory”, and the directory silently still shows one company, because the stale query is still in the box driving the filter.

7.2 The watchlist, and a real state bug

The star button on each card carries `data-watch="c1"`, and the delegated handler toggles membership of a `Set`:

```
1 const watchBtn = e.target.closest("[data-watch]");
2 if (watchBtn) {
3   const id = watchBtn.dataset.watch;
4   if (watchedIds.has(id)) watchedIds.delete(id); else watchedIds.add(id);
5   renderDirectory();      // <-- always the directory, never the watchlist
6   return;
7 }
```

Un-starring from the Watchlist screen does nothing visible

`companyCardHtml` is shared by both grids, so the Watchlist screen renders star buttons too. But the handler **unconditionally** calls `renderDirectory()` and never `renderWatchlist()`. So on the Watchlist screen:

1. You click the star to remove a company.
2. `watchedIds` correctly drops the id. The model is now right.
3. `renderDirectory()` re-renders a grid on a *hidden* screen.
4. The Watchlist grid in front of you is never re-rendered, so the star stays gold and the company stays on the list.

The user sees a dead button. Navigating away and back makes the change appear, which is the signature of exactly this class of bug: the model updated, the view did not. The fix is one line, re-rendering whichever grid is currently active, or simply calling both. This is the most likely bug for an interviewer to trip over by accident, because starring things and then visiting the watchlist to unstar one is the obvious way to explore the feature.

7.3 The star icon trick

```
1 ${icon("star", 15).replace('fill="none"', watched ? 'fill="currentColor"' : 'fill="none"')}
```

`icon()` returns an SVG string whose *first* `fill="none"` is on the `<svg>` element itself, so replacing the first occurrence flips the star between outlined and filled.

It works because of an accident of attribute order

`String.replace` with a string pattern replaces **only the first match**. This code depends on `fill="none"` appearing on the `<svg>` wrapper before it appears anywhere in the path data. That is true today. Reorder the attributes in `icon()`, or add an icon whose path contains `fill="none"`, and this silently paints the wrong element. A `fill` parameter on `icon()` would cost three lines and remove the landmine. Note also that the false branch replaces `fill="none"` with `fill="none"`, which is a no-op written out longhand.

8 The Company Profile: masking, seniority, and the unlock flow

8.1 Masking

Locked contact details are shown mangled rather than hidden, so the user can see that something real is there:

```

1 function maskEmail(email) {
2     if (!email) return "";
3     const [user, domain] = email.split("@");
4     return `${user[0]}\u2022\u2022\u2022\u2022\u2022@${domain}`;
5 }
6 function maskPhone(phone) {
7     if (!phone) return "";
8     return phone.slice(0, 3) + " \u2022\u2022\u2022\u2022-\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022";
9 }

```

Jargon: Why this listing shows \u2022

The real source contains the literal bullet character U+2022, typed directly. This document’s typesetter has no glyph for it and cannot print it, so each bullet is shown above as `\u2022`, which is JavaScript’s escape for exactly the same character. The two forms are interchangeable in real code: the listing above is valid JavaScript and behaves identically to what is in the file. Only the notation differs.

So `e.marsh@meridianagrotrade.example.com` renders as the first letter, five bullets, then the full domain; and `+55 11 4820-1187` renders as `+55` followed by a fixed bullet pattern. A CSS blur is layered on top via `.contact-row.is-masked`.

The mask leaks the domain, and the phone mask lies about length

Two things worth noticing:

- `maskEmail` preserves the **entire domain**. Since the domain is derived from the company name, and you are looking at that company's profile, that is not much of a leak here. But it means the mask hides only the local part.
- `maskPhone` emits a **fixed** pattern regardless of the real number's length. The real number is 15 characters; the mask is always the first 3 plus the same 12-character shape. It therefore does not preserve length, which is arguably better for privacy and worse for realism.
- `maskEmail` would produce the string `"undefined"` followed by bullets if handed an email with no local part, since `user[0]` on an empty string is `undefined` and template literals stringify it. The `if (!email)` guard catches `null` and `""` but not `"@domain.com"`. No such value exists in the fabricated data, so this cannot fire today.

The README claims these use “the same masking pattern as the real `maskValue()`”. That claim cannot be checked from this repository, since the real function is not here. **Treat it as inferred.**

8.2 The seniority classifier

Each contact gets a coloured badge derived from their job title by an ordered chain of regular expressions:

```

1 function getSeniorityBadge(designation) {
2   const d = (designation || "").toLowerCase();
3   if (/ceo|chief|director|owner|partner|president|founder)/.test(d))
4     return ["Decision Maker", "decision"];
5   if (/procurement/.test(d)) return ["Procurement", "procurement"];
6   if (/sales/.test(d)) return ["Sales", "sales"];
7   if (/manag|head)/.test(d))return ["Management", "management"];
8   return ["Team Member", "team"];
9 }

```

Note **manag** rather than **manager**: the truncated stem deliberately catches “Manager”, “Managing” and “Management” with one pattern. That is a nice touch.

Running it against all five fabricated contacts gives:

Designation	Badge	Matched on
Head of Export Sales	Sales	sales (before head is tested)
Managing Director	Decision Maker	director
Procurement Manager	Procurement	procurement (before manag)
Founder & CEO	Decision Maker	founder
Sales Director	Decision Maker	director (before sales)

First match wins, and the ordering produces two odd results

The chain is ordered by intended priority, and for the most part that is correct: a “Managing Director” should read as a decision maker, not as management, and the ordering delivers that.

But look at the two **sales** rows. “Sales Director” is badged **Decision Maker**, never Sales, because **director** is tested first. Meanwhile “Head of Export Sales” is badged **Sales**, not Management, because **sales** is tested before **head**. So of the two contacts with “Sales” in the title, the *more* senior one loses the Sales badge and the less senior one keeps it. Both outcomes are arguably defensible, but they are an emergent consequence of ordering rather than a decision anyone made, and the classifier can only ever return one badge where a real title genuinely carries two axes (function and seniority).

Of five fabricated contacts, three land on “Decision Maker”. The “Management” and “Team Member” branches are never exercised by the shipped data, so two of the five CSS badge colours are dead in practice.

8.3 The unlock flow

This is the centrepiece of the demo, and the thing the README is proudest of: the real application unlocks contacts **one at a time**, not per company, and the demo reproduces that.

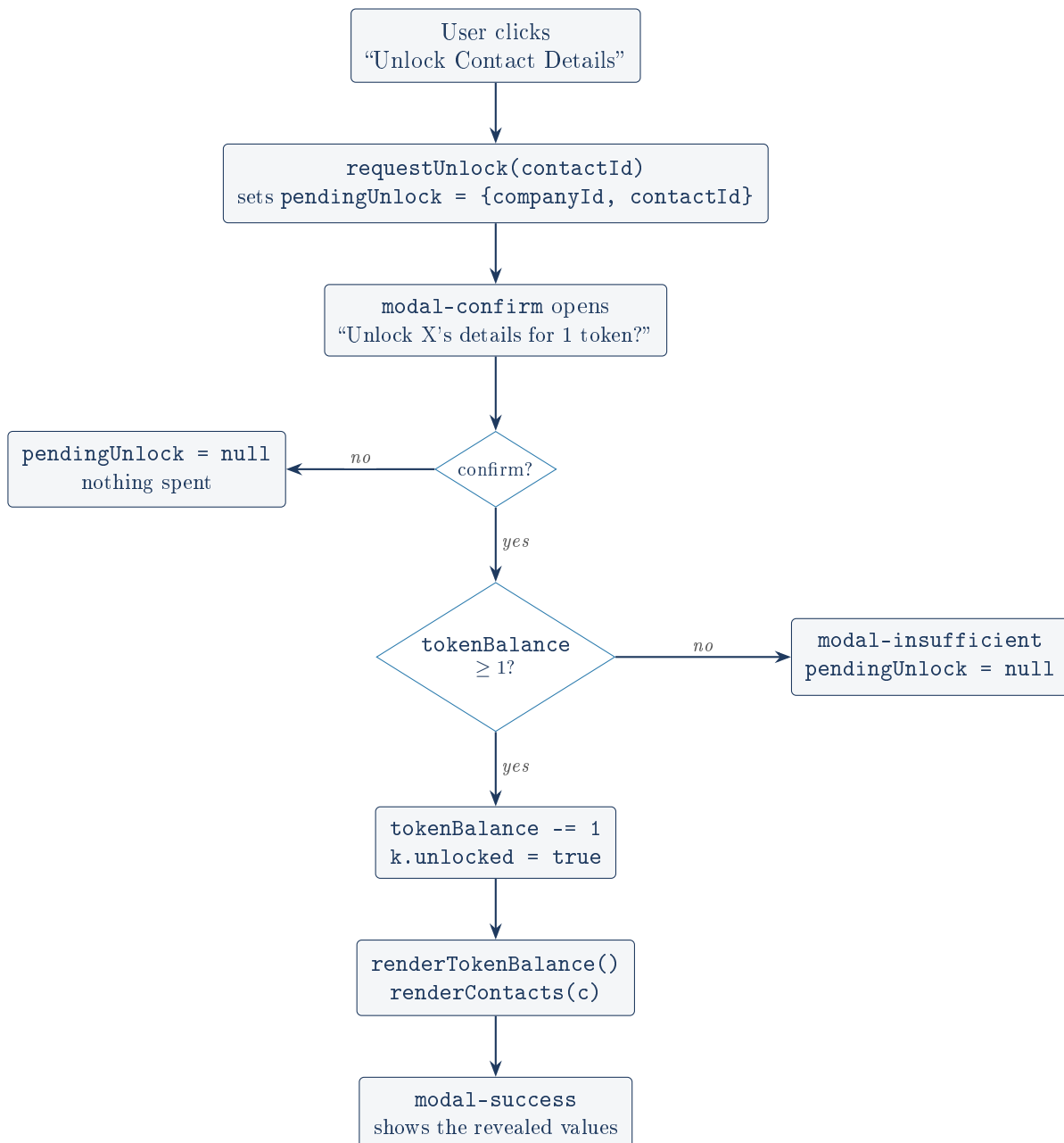


Figure 7: The per-contact unlock. Only the confirmed contact flips; every other contact on the page stays locked.

The `pendingUnlock` variable is what makes the modal work. The modal's OK button is a single, fixed element in the HTML; it cannot know which contact it is for. So `requestUnlock` stashes the target, and `confirmUnlock` picks it back up:

```

1 function confirmUnlock() {
2   hideModal("modal-confirm");
3   if (!pendingUnlock) return;
4   if (tokenBalance < 1) { showModal("modal-insufficient"); pendingUnlock = null; return; }
5   const c = COMPANIES.find(x => x.id === pendingUnlock.companyId);
6   const k = c.contacts.find(x => x.id === pendingUnlock.contactId);
7   tokenBalance -= 1;
8   k.unlocked = true;
9   renderTokenBalance();
10  renderContacts(c);
11  document.getElementById("success-text").textContent = `${k.name} \u00B7 ${k.phone} \u00B7 ${k.email}`;

```

```

12   showModal("modal-success");
13   pendingUnlock = null;
14 }

```

This function is careful in ways worth pointing out under questioning: it re-checks the balance at confirm time rather than trusting the check made when the button was drawn; it clears `pendingUnlock` on every exit path including the failure ones; and it mutates exactly one contact's `unlocked` flag before re-rendering the whole contact list from the model. The pattern (change the model, then re-render from the model) is the right one, and it is the same pattern the watchlist bug above gets wrong.

Two small holes in an otherwise solid flow

- **The confirm modal cannot be dismissed by clicking the backdrop or pressing Escape.** `.zl-modal-overlay` covers the screen and only `#confirm-cancel` closes it. Every modal on the page has this property. It is a one-listener fix and a standard expectation.
- **The success modal restates the data that is already on the page.** It writes name, phone and email into `success-text` immediately after `renderContacts` has already revealed exactly those values behind it. Harmless, but redundant.
- **Unlocking is irreversible without a reload**, since `unlocked` is mutated on the fixture itself. Demoing the locked state twice in one session is impossible.

8.4 The token economy, end to end

Event	Tokens	Consistent?
Starting balance	+7	
Unlock one contact	-1	yes: label, guard and deduction all say 1
Unlock a category	-5	no : the button says 500
Buy Starter (\$29/mo)	+20	
Buy Growth (\$79/mo)	+75	
Buy Enterprise (\$249/mo)	+300	

With four lockable contacts in the whole dataset and a starting balance of 7, a user can unlock *everything* and still have 3 tokens left. The scarcity the screen is built to dramatise never actually bites.

Buying a plan is an unconditional token faucet

```

1  const plan = SUBSCRIPTION_PLANS.find(p => p.id === buyPlanBtn.dataset.buyPlan);
2  tokenBalance += plan.tokens;
3  renderTokenBalance();
4  buyPlanBtn.innerHTML = `${icon("check-circle", 13)} Tokens added`;
5  setTimeout(() => { buyPlanBtn.textContent = "Get Plan"; }, 2000);

```

No payment, no state, no limit. Click “Get Plan” on Enterprise ten times and you have 3,000 tokens. There is no notion of a *current* plan at all: the plan cards are a button that adds a number. For a demo that is a reasonable shortcut, but the Subscription screen is the one screen where nothing is really being modelled.

Also note the two-second `setTimeout` restores the label with `textContent` after setting it with `innerHTML`. That is correct here, and mildly lucky: the icon SVG is written in as markup and then wiped by a plain-text assignment, which happens to be exactly what is wanted.

The insufficient-funds modal has a dead button

`#insufficient-buy` is labelled “Buy Tokens” and carries `data-view="subscription"`, so the delegated navigation handler fires and switches to the Subscription screen. But it *also* has its own listener, added later, that calls `hideModal`. Both run. The result is correct by accident: the modal closes and the view changes. The ordering works only because the delegated handler is on `<body>` and therefore runs after the element’s own listener during bubbling. It is fragile, and the `data-view` attribute alone would have done the whole job if `hideModal` were called from the navigation branch.

9 Presentation: icons, fonts and colour

9.1 The icon set

`icons.js` is a single function over a lookup table of 20 SVG path fragments:

```
1 function icon(name, size) {
2   size = size || 16;
3   const paths = { "search": '<circle cx="11" cy="11" r="8"/><path d="m21 21-4.3-4.3"/>', /* ...
4     */ };
5   const body = paths[name] || "";
6   return `<svg width="${size}" height="${size}" viewBox="0 0 24 24" fill="none"
7     stroke="currentColor" stroke-width="2" stroke-linecap="round"
8     stroke-linejoin="round" aria-hidden="true">${body}</svg>`;
9 }
```

Jargon: SVG, stroke, and currentColor

SVG draws pictures with maths rather than pixels, so an icon stays sharp at any size. **stroke** is the colour of the line; **fill** is the colour inside it. These icons are outlines, so they set `fill="none"` and stroke only. **currentColor** is a keyword meaning “whatever the CSS text colour is here”, which is why one icon definition works on a dark navbar and a white card without being redefined.

Every icon shares Lucide’s 24 by 24 `viewBox`, 2px stroke and round caps, which is the whole point: the real application uses Lucide, and matching those three parameters is what makes a hand-drawn icon read as belonging to the same family.

An unknown icon name fails silently, and a third of the set is unused

`paths[name] || ""` means `icon("serch")` returns a valid, empty, invisible SVG. No error, no warning, just a gap. A `console.warn` in the fallback would cost one line. Of the 20 icons defined, several (`shield-check`, `filter`, `x`, `moon`, `chevron-right`, `arrow-right`, `zap`, `chevron-down`, `bar-chart`, `search`) are never called through `icon()`. Some are genuinely unused; others exist twice, because `index.html` pastes the *same* SVG markup inline rather than calling the function. The moon, the zap and the chevron in the navbar are all hand-copied into the HTML while also sitting in the lookup table. There is one icon set maintained in two places.

9.2 The font, and a stale README

`styles.css` self-hosts a variable font:

```
1 @font-face {
2   font-family: 'Inter';
3   src: url('assets/fonts/inter-variable.woff2') format('woff2-variations'),
```

```

4     url('assets/fonts/inter-variable.woff2') format('woff2');
5     font-weight: 100 900;
6     font-display: swap;
7 }

```

Jargon: Variable font, and font-display: swap

An ordinary font ships one file per weight. A **variable font** ships one file that can be rendered at any weight on a continuous axis, which is what `font-weight: 100 900` declares. `font-display: swap` tells the browser to draw text immediately in a fallback font and swap Inter in when it arrives, so the page is never blank while waiting.

The real ZaraiLink loads Inter from Google Fonts. This demo self-hosts it instead, so the page stays fully offline and makes no external request. Listing the same file twice with two `format()` hints is a deliberate fallback for browsers that do not understand `woff2-variations`.

README.md contradicts the code about the font

The README's Architecture section states:

“The real app loads ‘Inter’ from Google Fonts; this demo stays fully self-contained and offline, so it falls back to the closest system font instead of fetching that externally.”

That is **no longer true**. The demo does not fall back to a system font: it ships `assets/fonts/inter-variable.woff2` and uses the real Inter. The git history shows why. The commit “Self-host real Inter variable font and replace emoji with Lucide-style SVG icons” came *after* the README paragraph was written, and the paragraph was never updated. `styles.css`'s own header comment is correct and says the demo self-hosts. So the repository now describes its own font strategy two different ways in two files, and the README is the wrong one.

The code wins. The README should be corrected. This is exactly the kind of detail a careful reviewer checks, and being caught claiming a system-font fallback while shipping a 48 KB font file is a bad look for a project whose entire pitch is honesty about what is real.

The font file is 48 KB, which is small for a full variable Inter (the complete Latin range typically runs several times that). It is therefore *probably* a subset containing only the glyphs this page needs. **This is inferred from the file size and has not been confirmed by inspecting the font tables.**

9.3 Colour tokens

`styles.css` declares the palette as CSS custom properties, which its header comment states were copied 1:1 from the real application's `index.css :root` block:

```

1 :root {
2   --primary: #10B981;      /* emerald: the brand colour */
3   --primary-hover: #059669;
4   --slate-900: #0F172A;    /* the navbar and stats strip */
5   --text: #1A1A1A;
6   --border: #E5E7EB;
7   --r-card: 16px; --r-btn: 8px; --wrap: 1200px;
8 }

```

Jargon: CSS custom properties (variables)

A name starting with two hyphens declares a reusable value. `var(--primary)` then substitutes it anywhere. Change the one declaration and every use updates. These are usually called **design tokens** when they encode a design system's decisions rather than incidental values.

One token is referenced but never declared

`.zl-user-menu__name` reads `var(--slate-200, #e2e8f0)`. There is no `--slate-200` in the `:root` block, so the fallback always wins. It renders correctly (that fallback is the right colour) but the variable is a fiction. The palette declares `slate-900`, `800`, `700` and `400`, and `200` was either forgotten or dropped.

Similarly the `.zl-theme-toggle` rule styles a moon button with hover states, and `index.html` renders that button with an `aria-label` of "Toggle dark mode", but **no JavaScript is attached to it and no dark theme exists in the stylesheet**. Clicking it does nothing at all. Same for the two quick pills "I want to buy" and "I want to sell", which have no `data-view` and no handler; the two beside them do navigate, which makes the dead pair harder to spot, not easier. And the `.zl-user-menu` is styled with `cursor: pointer` and a hover background, advertising a dropdown that does not exist.

That is four visible controls (theme toggle, two quick pills, user menu) plus the sidebar checkboxes and the scope toggle that respond to the mouse and do nothing. For a demo whose purpose is to be clicked by strangers, dead controls are the highest-cost defect class in the repository: each one is an invitation for an interviewer to find the seam.

10 Security, licensing and deployment

10.1 XSS, and why it does not matter here

Nearly everything is rendered by building HTML strings and assigning them to `innerHTML`. There is an `escapeHtml` helper, and it is used on most interpolated values:

```
1 function escapeHtml(str) {
2   return String(str == null ? "" : str)
3     .replace(/&/g, "&amp;").replace(/</g, "&lt;").replace(/>/g, "&gt;");
4 }
```

Jargon: XSS (cross-site scripting)

If an application takes text from somewhere and drops it straight into the page as HTML, then text containing `<script>` becomes a real, executing script. That is **XSS**. The defence is escaping: turning `<` into `<`; so the browser draws it as a character instead of interpreting it as markup.

The escaping is incomplete, and safe only because the data is a constant

`escapeHtml` handles `&`, `<` and `>`, but **not** quotes. Several interpolations land *inside* HTML attributes, for example `data-open-profile="${c.id}"`. A value containing a double quote would break out of the attribute, and no amount of `<`-escaping stops it. Escaping `"` and `'` is the standard completion of this function.

Several other values skip `escapeHtml` entirely (`c.shipments`, `p.tokens`, the seniority label, the contact-summary string), and `icon()` output is injected raw by design.

None of this is exploitable, and it is important to be precise about why: every string in this application is a hard-coded literal in `demo-data.js`. There is no user-supplied data, no

network input, no URL parameter, and no storage. The two text inputs are read but never rendered back as HTML. The attack surface is empty.

So this is not a vulnerability. It is a habit that would become one the instant this code met real data, which is precisely what the real ZaraiLink does. The right answer to “is this safe?” is: yes here, for a reason that would not survive contact with a backend.

A full scan of every JavaScript, HTML, CSS and configuration file in the repository for API keys, tokens, passwords, bearer credentials, private keys and connection strings returned **nothing**. There are no secrets in this repository, which is what you would expect from a codebase with no network calls, but it was checked rather than assumed. Every domain in the fabricated data sits under `.example.com`, which is reserved by RFC 2606 and can never resolve.

10.2 Deployment

```
1 [build]
2   publish = "."
```

The whole of `netlify.toml`. It declares no build command, because there is nothing to build: Netlify serves the folder verbatim. This is the payoff for having skipped a toolchain.

`index.html` also carries `<meta name="robots" content="noindex">`, which asks search engines not to index the page. That is a deliberate and correct choice: a page full of invented companies with invented contact details should not turn up in a search for those names.

10.3 Licence

The repository is under **PolyForm Strict 1.0.0**. This is worth understanding because it is unusual and it is not an open-source licence.

What PolyForm Strict actually permits

It grants permission to *read* the source and nothing else. No use, no modification, no distribution, no derivative works. It is source-available, not open source.

For portfolio work this is a coherent choice: the point is to let people read the code and judge it, not to donate it. It is especially coherent *here*, where the underlying product is a private team project. But it should not be described as open source, and the copyright line reads “Copyright 2026 Salman Adnan” on a repository whose subject matter belongs to three people. The demo’s code is Salman’s alone, so that is defensible; the distinction between “this demo’s code” and “ZaraiLink” is doing real work in that line.

11 Consolidated findings

Everything below was verified by reading the source and, where it involved logic, by executing it.

11.1 Real defects

Defect	Where	Effect
Category unlock says 500 tokens, charges 5	<code>app.js selectPill</code>	Label and code disagree 100-fold
Un-starring on the Watchlist re-renders the Directory	<code>app.js data-watch</code> branch	Button looks dead to the user
“Product Search” mode unreachable	<code>detectMode(q, true)</code> at both call sites	A documented feature cannot occur
Empty state hard-codes “No suppliers found.”	<code>renderDirectory</code>	Wrong noun in Buyer mode
Directory search box never cleared	<code>renderDirectory</code>	Stale filter on return from a profile
Shipment chip shows 56, data sums to 125	<code>COMPANIES.length * 14</code>	Self-contradicting headline
<code>escapeHtml</code> does not escape quotes	<code>escapeHtml</code>	Latent; not exploitable today

11.2 Dead code and dead controls

Thing	Status
<code>HS_TREE</code>	Declared in <code>demo-data.js</code> , referenced nowhere
<code>company.website</code> , <code>company.last_active</code>	Populated, never rendered
<code>currentScope</code>	Written on every scope click, read nowhere
Import/Export toggle	Moves a highlight, changes no behaviour
Theme toggle (moon)	Styled and labelled, no listener, no dark theme
“I want to buy” / “I want to sell” pills	No <code>data-view</code> , no handler
User menu	<code>cursor: pointer</code> , hover state, no dropdown
“Refine by Product” checkboxes	Real counts, no listener
<code>--slate-200</code>	Used with a fallback, never declared
10 of 20 icons in <code>icons.js</code>	Never called; several duplicated inline in HTML
<code>.mode-pill.is-product</code>	Styled for a state that cannot occur
Management / Team Member badges	No fabricated contact reaches those branches

11.3 Where the README overstates the code

1. **Font.** README says the demo falls back to a system font. It self-hosts Inter. The code is right; the README is stale.
2. **Search modes.** README says the pill flips between three modes. Only two are reachable.
3. **Trade-direction pills.** README is accurate (“filter by country”), but the screen presents four pills that behave as two.

11.4 What the code does well

It is worth being equally precise about this, because the list is real:

- **Event delegation from `<body>`.** The correct architecture for an `innerHTML`-driven UI, and the reason re-rendering is free.

- **Per-contact unlock state.** The README calls this out as a deliberate choice over a simpler per-company toggle, and the code delivers it. `pendingUnlock` plus a confirm-time re-check of the balance is the right shape for a two-step transaction.
- **Change the model, then re-render from the model.** `confirmUnlock` does this correctly. The watchlist bug is a deviation from a pattern the codebase otherwise gets right.
- **Filters stored as functions on the pill data.** `selectPill` stays generic.
- **No build step.** A demo whose whole job is to still work in two years, opened from a file path by a stranger, is exactly the case where a toolchain is a liability.
- **The honesty scaffolding is real and everywhere.** A yellow banner at the top of every screen, a footer disclaimer, `noindex`, `.example.com` domains, and “(demo)” and “fabricated” printed into the results screen itself. This is not an afterthought.

11.5 The most likely interview questions

1. *“Show me the search working.”* Type two different queries. The results do not change. Lead with this rather than being caught by it.
2. *“What does the Import/Export toggle do?”* Nothing. It is wired to a variable nothing reads.
3. *“Why does this say 500 tokens and take 5?”* A bug. Fix it before the interview.
4. *“Which parts of ZaraiLink did you write?”* The README says 64 of 97 commits, and this repository cannot back that up. Have the private repository’s history ready, and name Umar Kashif and Fahad Nadeem before being asked.
5. *“Is this the real product?”* No. Every screen is a reconstruction and every value is invented.

The highest-value hour of work on this repository

Not a new feature. Three things, in order:

1. **Fix the 500/5 token label** and the watchlist re-render. Both are one-line fixes and both are things a clicking interviewer will find.
2. **Correct the README’s font paragraph** and soften the three-modes claim. A project sold on honesty cannot afford a README that overstates its own code, however innocently it drifted.
3. **Either wire up or delete the dead controls.** The theme toggle, the two quick pills, the scope toggle and the sidebar checkboxes. Deleting them costs nothing and removes six chances for the demo to look broken rather than scripted. A demo with fewer controls that all work is strictly stronger than one with more controls where half are scenery.

The README’s own “What I’d do differently” proposes adding the Trade Intelligence cluster (link prediction, node2vec, the GNN work). That is the right *ambition* and the wrong *next step*: adding a sixth screen to a demo with six dead controls makes the seam wider, not narrower.